

Assignment #8: 田忌赛马来了

2024 fall, Compiled by 徐至晟, 光华

说明:

- 1) 请把每个题目解题思路 (可选), 源码Python, 或者C++ (已经在Codeforces/Openjudge上AC), 截图 (包含Accepted), 填写到下面作业模版中 (推荐使用 typora <https://typoraio.cn>, 或者用 word)。AC 或者没有AC, 都请标上每个题目大致花费时间。
- 2) 提交时候先提交pdf文件, 再把md或者doc文件上传到右侧“作业评论”。Canvas需要有同学清晰头像、提交文件有pdf、“作业评论”区有上传的md或者doc附件。
- 3) 如果不能在截止前提交作业, 请写明原因。

1. 题目

12558: 岛屿周长

matrices, <http://cs101.openjudge.cn/practice/12558/>

思路:

1的格子对周长的贡献=其周围有几个0

居然第二层循环把m+1写成n+1。。。害的调试半天

代码:

```
n,m=map(int,input().split())
a=[[0 for _ in range(m+2)]]
for _ in range(n):
    s=[0]+list(map(int,input().split()))+[0]
    a.append(s)
a.append([0 for _ in range(m+2)])
ans=0
for i in range(1,n+1):
    for j in range(1,m+1):
        if a[i][j]==1:
            cnt=a[i-1][j]+a[i][j-1]+a[i][j+1]+a[i+1][j]
            ans+=4-cnt
print(ans)
```

代码运行截图 (至少包含有"Accepted")

#47210010提交状态

状态: Accepted

洛谷

LeetCode54.螺旋矩阵

matrice, <https://leetcode.cn/problems/spiral-matrix/>

与OJ这个题目一样的 18106: 螺旋矩阵, <http://cs101.openjudge.cn/practice/18106>

思路:

是一个走地图的过程, 如果碰到边界或者下一格已经走过, 则转向。可以用方向数组 $dx[]$, $dy[]$ 和参数 $a \in \{0, 1, 2, 3\}$ 得到目前行走的方向, 拐角处令 a 为 $(a + 1) \bmod 4$ 。

代码:

```
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int m=matrix.size();
        int n=matrix[0].size();
        int cnt=0,x=0,y=0,a=0,dy[4]={1,0,-1,0},dx[4]={0,1,0,-1};
        vector<int> ans;
        bool v[103][103];
        while(cnt<m*n)
        {
            v[x][y]=true;
            ans.push_back(matrix[x][y]);
            if (x+dx[a]<0 || x+dx[a]>=m || y+dy[a]<0 || y+dy[a]>=n || v[x+dx[a]]
[y+dy[a]])
                a=(a+1)%4;
            x+=dx[a];
            y+=dy[a];
            cnt+=1;
        }
        return ans;
    }
};
```

代码运行截图 (至少包含有"Accepted")

通过



melocotón 提交于 2024.11.13 17:20

[官方题解](#)[写题解](#)

洛谷

04133:垃圾炸弹

matrices, <http://cs101.openjudge.cn/practice/04133/>

思路:

枚举所有放炸弹的位置 (x, y) ($x, y \in [0, 1024] \cap \mathbb{N}$), 看每一堆垃圾是否能被覆盖到。输出第一个数字是放炸弹位置的方案数。时间复杂度 $\Theta(m^2n)$, 其中 $m = 1025$ 为地图的边长。

代码:

```
import math
d=int(input())
n=int(input())
a=[]
for _ in range(n):
    a.append(tuple(map(int,input().split())))
num=0
clean=0
for x in range(1025):
    for y in range(1025):
        cnt=0
        for e in a:
            if abs(x-e[0])<=d and abs(y-e[1])<=d:
                cnt+=e[2]
        if cnt>clean:
            clean=cnt
            num=1
        elif cnt==clean:
            num+=1
print(num,clean)
```

代码运行截图 (至少包含有"Accepted")

#47210415提交状态

状态: Accepted

源代码

```
import math
d=int(input())
```

洛谷

LeetCode376.摆动序列

greedy, dp, <https://leetcode.cn/problems/wiggle-subsequence/>

与OJ这个题目一样的, 26976:摆动序列, <http://cs101.openjudge.cn/routine/26976/>

思路:

本题还是更习惯用DP写。记 $f[i], g[i]$ 分别表示子序列以 $nums_i$ 结尾, 末尾两个数成递增/递减关系时最长摆动子序列的长度。寻找前面 $nums_j$ 作为倒数第二个数, 依照合法性对两个DP数组更新。时间复杂度 $\Theta(n^2)$ 。

代码:

```
n=int(input())
a=list(map(int,input().split()))
f=[1]
g=[1]
ans=1
for i in range(1,n):
    f.append(0)
    g.append(0)
    for j in range(0,i):
        if a[j]>a[i]:
            g[i]=max(g[i],f[j]+1)
        elif a[j]<a[i]:
            f[i]=max(f[i],g[j]+1)
    ans=max(ans,f[i],g[i])
print(ans)
```

代码运行截图 (至少包含有"Accepted")

24n2400015881(David_X)

26976: 摆动序列

Accepted

3664kB 230ms

296 B

Python3

4分钟前

CF455A: Boredom

dp, 1500, <https://codeforces.com/contest/455/problem/A>

思路:

无清晰思路, 参考他人题解

来源: 洛谷

$Dp[i]$ 表示算到 i 这个数时的最大分数为多少。

那么该如何转移呢, 我们知道取一个 x 后, 这些 $num[x + 1]$ 与 $num[x - 1]$ 会被删除, 所以就可以很容易的得到 $dp[i] = \max(dp[i - 1], dp[i - 2] + num[i] * i);$

初始值: $dp[1] = num[1];$

洛谷

代码:

```
n=int(input())
a=list(map(int,input().split()))
cnt=[0 for _ in range(100003)]
for x in a:
    cnt[x]+=1
f=[0 for _ in range(100003)]
f[1]=cnt[1]
ans=f[1]
for i in range(2,100001):
    f[i]=max(f[i-1], f[i-2]+i*cnt[i])
    ans=max(ans, f[i])
print(ans)
```

代码运行截图 (至少包含有"Accepted")

292078404

Nov/18/2024 01:04 UTC+8

Ebwje

A - Boredom

Python 3

Accepted

186 ms

13000 KB

02287: Tian Ji -- The Horse Racing

greedy, dfs <http://cs101.openjudge.cn/practice/02287>

思路:

错误的贪心思路: 田忌的马从强到弱考虑, 尽量赢下齐王能被赢下的最强的马。

问题应该在于战平的处理。我的方法是: 比较当前两队剩下的第二名; 若下一轮田忌能赢, 这一轮选择打平; 若下一轮赢不了 (注意是负或平, 不只是负) 而第一名的马有机会赢对面的第二名, 则拿下对面第二名, 将第一名留给最差的一匹马解决; 若田第一=王第一=王第二, 则记一局平, 仍然将第一名留给最差的一匹马解决 (这个在代码中没有体现, 只用了头指针, 没有尾指针)

直到看到了群里的这组数据。9-7 8-6 6-3 2-2 1-9 得400，输出200。原来9-9 8-7 6-3 2-2 1-6 强者保平不是最佳策略。所以大多数题解是从最弱的两匹马考虑起，而不是从强的马开始。

```
5
1 2 6 8 9
2 3 6 7 9
```

代码 (WA1) :

```
n=int(input())
while n:
    a=list(map(int,input().split()))
    b=list(map(int,input().split()))
    a.sort(reverse=True)
    b.sort(reverse=True)
    i=0
    j=0
    win=0
    draw=0
    while i<n:
        while j<n and a[i]<b[j]:
            j+=1
        if j>=n:
            break
        if a[i]>b[j]:
            win+=1
        elif a[i]==b[j]:
            if i<n-1 and j<n-1:
                if a[i+1]>b[j+1]:
                    draw+=1
                elif a[i+1]<b[j+1]:
                    win+=1
                j+=1
            else:
                draw+=1
        else:
            if j==n-1:
                draw+=1
            else:
                if b[n-1]<a[n-1]:
                    win+=1
                elif b[n-1]==a[n-1]:
                    draw+=1

        j+=1
        i+=1
    print((2*win+draw-n)*200)
    n=int(input())
```

错误2：仍然从强到弱，赢下能击败的最强的，赢了才算数，标记配对关系。剩下未标记（不能完胜）的强配弱，尽量增加平局对数。仍然WA

```
n=int(input())
while n:
```

```

a=list(map(int,input().split()))
b=list(map(int,input().split()))
u=[-1 for _ in range(n)]
v=[-1 for _ in range(n)]
a.sort(reverse=True)
b.sort(reverse=True)
i=0
j=0
win=0
draw=0
while i<n:
    while j<n and a[i]<=b[j]:
        j+=1
    if j>=n:
        break
    if a[i]>b[j]:
        win+=1
        u[i]=j
        v[j]=i
        j+=1
        i+=1
    j=n-1
for i in range(n):
    if u[i]>0:
        continue
    while j>=0 and v[j]>=0:
        j-=1
    if a[i]==b[j]:
        draw+=1
        u[i]=j
        v[j]=i
print((2*win+draw-n)*200)
n=int(input())

```

代码运行截图 (至少包含有"Accepted")

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“计概2024fall每日选做”、CF、LeetCode、洛谷等网站题目。

Matrices 题目可练习Python二维数组的初始化。

田忌赛马比较有难度，属于贪心和DP都比较尴尬的题目，贪心细节多，出错可能性大，DP状态方程不自然。

以前DP数组习惯建立二维下标，特别是带有0/1信息的，有时会使思路复杂化，之后可以删减改进。